# NetPay Manager: plan de escalabilidad y evolución tecnológica

**Fase 3 — Documentación para escalabilidad futura**
**Empresa destinataria:** Gravity
**Alcance:** procesos futuros, evolución del modelo de datos, infraestructura y ampliación de recibos a impresión térmica.

## 1. Propósito y contexto del negocio

NetPay Manager centraliza la administración interna de clientes, planes de internet, empleados cobradores, obligaciones mensuales, pagos y recibos. Su origen es la sustitución del control disperso en hojas de cálculo por registros relacionados y trazables. Los empleados acceden al sistema; los clientes son registros comerciales y no necesitan cuentas propias.

El crecimiento del negocio aumenta simultáneamente el número de clientes, los cargos acumulados por mes, las operaciones concurrentes de cobro, las consultas históricas y los comprobantes solicitados. Escalar la solución requiere preservar la explicación de los saldos y la continuidad del servicio, además de ampliar recursos de cómputo.

Este documento reúne el punto de partida, la arquitectura objetivo, tres procesos futuros diagramados en BPMN, las migraciones necesarias y las tecnologías propuestas con sus condiciones de adopción. Incluye el diseño de una integración térmica pendiente. Las propuestas se describen como evolución futura; su presentación no acredita que estén desplegadas ni que se hayan realizado pruebas con equipos no disponibles.

### 1.1 Necesidades que se conservan del planteamiento inicial

| **Necesidad de Gravity** | **Criterio que debe preservarse al crecer** |
|---|---|
| Registrar pagos parciales, completos y adelantados | Separar dinero recibido, obligación mensual y aplicación del importe |
| Mantener historia ante cambios de plan/precio | Conservar cargos existentes y documentar la tarifa efectiva de obligaciones nuevas |
| Distribuir clientes entre cobradores | Comprobar permisos y pertenencia del recurso en el servidor |
| Consultar cartera, reportes y auditoría | Mantener filtros, fecha de corte y trazabilidad sin bloquear el cobro diario |

| Necesidad de Gravity | Criterio que debe preservarse al crecer |
|---|---|
| Entregar comprobantes | Reutilizar un recibo único del pago en PDF y, posteriormente, en papel térmico |
| Recuperar información ante fallos | Disponer de copias externas, restauraciones probadas y tiempos de recuperación medidos |

El diseño inicial contemplaba frontend estático en servicios administrados y PostgreSQL en RDS. La instalación materializada utiliza EC2 y contenedores. La evolución propuesta retoma servicios administrados cuando exista una necesidad operativa y presupuesto; su sola adopción no resuelve errores de integridad ni sustituye la validación funcional.

# 2. Situación actual y límites de crecimiento

## 2.1 Componentes implementados

- **Frontend:** Angular 19 con componentes standalone, rutas de carga diferida y acceso REST. Se publica como contenido estático servido por Caddy en una EC2 independiente.
- **Backend:** API Java 17/Spring Boot, organizada en dominio, aplicación e infraestructura. Centraliza reglas, autorización, transacciones y acceso a datos.
- **Persistencia:** PostgreSQL 16 en un contenedor del mismo host que la API. Flyway versiona el esquema mediante V1–V21 y Hibernate valida su correspondencia con las entidades.
- **Identidad:** JWT con credencial persistida, estado del empleado y permisos efectivos consultados en la base. Existe segundo paso opcional por correo. No se depende de una sesión HTTP en memoria de una sola API.
- **Integraciones:** Gmail API para correo y Cloudinary para archivos/recibos, según configuración del entorno.
- **Entrega:** imágenes Docker etiquetadas por SHA, despliegue con GitHub Actions/ECR/SSM e infraestructura declarada con Terraform.

El estado aplicado de migraciones y la imagen activa se comprueban en cada ambiente. La existencia de una funcionalidad en el repositorio no garantiza que la versión publicada la contenga o que una integración externa tenga credenciales operativas.

## 2.2 Límites que deben resolverse

| Situación actual | Efecto si aumenta el uso | Evolución necesaria |
|---|---|---|
| API y PostgreSQL comparten host | Un fallo de la instancia interrumpe ambos; compiten por memoria/disco | Separar base y API, proteger datos y medir capacidad |
| Parte de las consultas asegura cargos faltantes antes de leer | Una consulta puede producir escrituras y trabajo creciente sobre la cartera | Generación programada por lotes con control de completitud |
| No hay historial completo de contratación/tarifa | Un cargo antiguo aún no generado toma el plan/precio disponible al crearse | Historial con vigencia y origen verificable de importes |
| Distribución de pagos sin control uniforme de concurrencia | Dos operaciones simultáneas pueden calcular sobre el mismo saldo | Bloqueo o versionado y pruebas de cobros concurrentes |
| Exportación síncrona de todo el filtro | Aumentan tiempo HTTP, memoria de render y competencia con cobros | Solicitudes persistidas y trabajadores independientes |
| Dependencias externas dentro de operaciones | Su latencia/fallo afecta la interacción; SQL no revierte el efecto remoto | Outbox, reintentos controlados y estado de entrega |
| Controles por registro y contratos públicos aún no uniformes | Añadir consumidores puede ampliar inconsistencias de autorización | Verificar pertenencia e identificar recursos por UUID público |
| Healthchecks y logs locales sin alertado centralizado | Un fallo puede detectarse tarde y perder contexto entre instancias | Métricas, logs centralizados, alarmas y correlación |
| Volumen EBS sin respaldo externo/PITR automatizado acreditado | Persistencia local sin recuperación garantizada ante pérdida del host | Copias externas, retención y simulacros |

La cuenta actual también puede incluir pagos no completados en la suma usada para mostrar crédito, aunque solo los completados generan aplicaciones. Antes de aumentar la carga o automatizar compensaciones debe alinearse ese cálculo con el dinero confirmado. Las mejoras de capacidad parten de reglas financieras conciliadas.

### 2.3 Medición y criterios para decidir

Se debe levantar una línea base con volumen de clientes, cargos, pagos, recibos y auditoría; usuarios concurrentes; tiempos P50/P95; errores; uso de CPU/memoria/disco; conexiones SQL y duración de consultas. P95 expresa el tiempo que no supera el 95 % de las solicitudes observadas. La medición debe separar lectura, cobro, importación y exportación.

| **Señal observada** | **Decisión posible** | **Cómo comprobar el resultado** |
|---|---|---|
| Cobros más lentos durante exportaciones | Aislar generación documental | Comparar latencia del cobro con y sin exportaciones concurrentes |
| Consultas SQL lentas o muchas conexiones | Optimizar consulta, índice y pool antes de replicar API | Planes de ejecución, esperas SQL y carga representativa |
| Saturación sostenida de API | Ajustar recursos; después incorporar réplicas | Prueba de carga y operación al retirar una réplica |
| Generación incompleta o tardía de cargos | Ejecutar P-E01 y medir cobertura | Clientes elegibles frente a cargos generados e incidencias |
| Tiempo excesivo de recuperación | Mejorar respaldo, restauración y ubicación de datos | Simulacro con tiempos y conciliación documentados |
| Más puntos de cobro físicos | Pilotar P-E03 con dispositivos registrados | Disponibilidad, duplicados y resultados ambiguos de impresión |

No se afirma una capacidad fija de usuarios ni se presentan tiempos objetivo como garantías medidas. Los límites de aceptación se acuerdan con Gravity usando la línea base y la duración tolerable de una interrupción.

## 3. Arquitectura objetivo y estrategia de evolución

### 3.1 Separación progresiva de responsabilidades

La API puede conservar su arquitectura de monolito modular. La primera separación es operativa: servir contenido estático, atender transacciones y ejecutar tareas largas en recursos

que puedan dimensionarse de forma independiente. No es necesario convertir cada módulo en un microservicio.

| **Componente objetivo** | **Responsabilidad y comunicación** |
|---|---|
| Angular en almacenamiento estático con CDN | Distribuir la interfaz por HTTPS; consumir API y consultar estados de trabajos |
| Balanceador y réplicas de API | Autorizar, ejecutar transacciones breves y registrar solicitudes; usar configuración externa y datos compartidos |
| PostgreSQL administrado en red privada | Fuente de verdad para cartera, pagos, aplicaciones y trabajos; respaldos y recuperación configurados |
| Planificador | Iniciar ejecuciones periódicas con una clave única por tipo y período |
| Outbox y despachador | Conservar eventos junto con el cambio SQL y publicarlos después de confirmar la transacción |
| Cola y trabajadores | Procesar reportes/cargos/documentos con concurrencia limitada, idempotencia y reintentos |
| Almacenamiento privado de documentos | Conservar archivos con expiración/retención y acceso autorizado |
| Observabilidad central | Relacionar solicitud, trabajo, ejecución, error y versión desplegada |
| Agente del punto de cobro | Acceder a la impresora local sin exponerla directamente a Internet |

Un evento outbox es un registro pendiente de despacho creado en la misma transacción que la solicitud de negocio. Si la publicación se repite, el consumidor debe reconocer el identificador del trabajo y evitar duplicar su efecto. Se asume entrega **al menos una vez**, no entrega exactamente una vez.

## 3.2 Orden de adopción

1. **Medir y estabilizar:** corregir sumas financieras y alcance por registro, revisar consultas e introducir observabilidad.
2. **Proteger y separar los datos:** copias externas verificadas, simulacro y migración a una base independiente cuando se apruebe.

3. **Preparar integridad:** historial tarifario, concurrencia de pagos y migraciones compatibles.
4. **Automatizar cargas:** generación de cargos y reportes mediante los procesos P-E01 y P-E02.
5. **Escalar API y frontend:** balanceador/réplicas y CDN después de comprobar consistencia y capacidad de la base.
6. **Ampliar operación física:** piloto térmico P-E03 condicionado a compra y validación del dispositivo.

Los pasos pueden dividirse en entregas pequeñas. Cada liberación necesita una validación funcional y de recuperación. La impresión térmica no es un requisito previo para escalar la API ni para mantener disponible el recibo digital.

# 4. Procesos futuros identificados y diagramados

## 4.1 Convenciones de lectura BPMN

Se conserva la organización visual por participantes y responsabilidades del modelado inicial: eventos circulares, tareas con esquinas redondeadas y decisiones en rombos. Los nuevos modelos son descriptivos; no requieren implantar un motor de procesos.

- **Círculo de borde fino:** inicio; un reloj indica disparo por calendario.
- **Círculo de borde grueso:** fin de ese flujo.
- **Rombo con X:** decisión exclusiva; se sigue una única salida según su condición.
- **Rectángulo con signo +:** subproceso resumido; un evento de doble borde con rayo en su límite captura un error y conduce a recuperación.
- **Tarea U / S:** actividad de usuario o de servicio automatizado.
- **Pool:** participante independiente; sus carriles distribuyen responsabilidades internas.
- **Flecha continua:** secuencia dentro del mismo participante, incluso entre carriles.
- **Línea discontinua con origen circular y flecha abierta:** mensaje entre participantes; no representa una transacción compartida.

Cada figura se acompaña de objetivo, entradas, reglas, excepciones y aceptación, de modo que el proceso se entiende con el contenido de este documento. Para composición del PDF se utilizan los tres marcadores siguientes; el nombre de archivo identifica únicamente el recurso gráfico que debe insertarse.

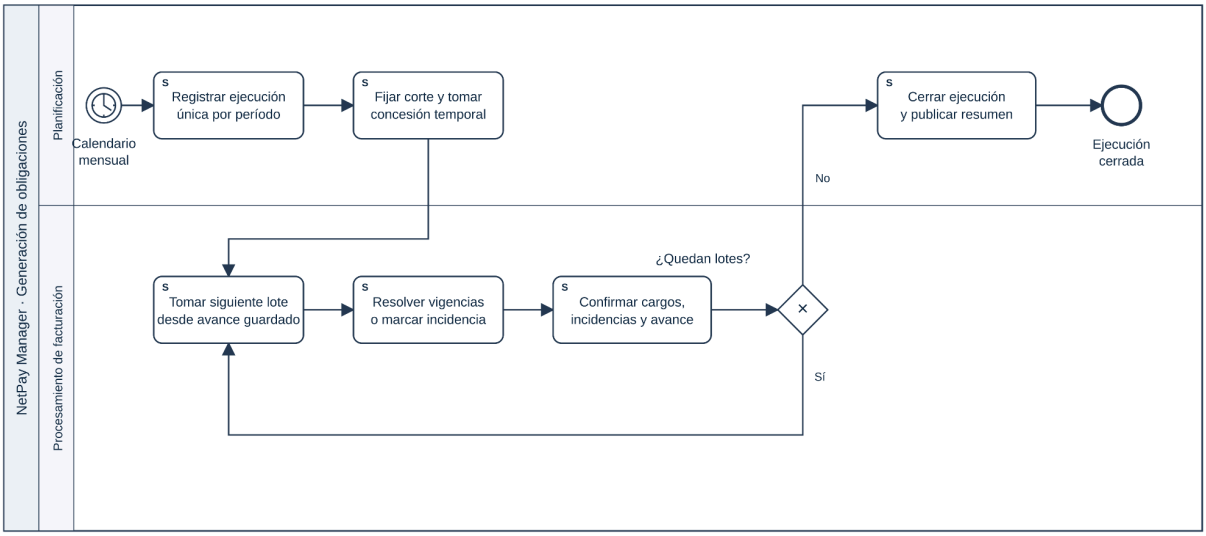
## 4.2 P-E01 — Generación programada de cargos con tarifas históricas

**Objetivo:** tener las obligaciones mensuales completas antes de su consulta, sin recalcular importes ya establecidos y sin concentrar toda la cartera en una sola transacción.

**Participante:** NetPay Manager. **Carriles:** planificación y procesamiento de facturación. **Disparador:** calendario mensual o recuperación controlada de una ejecución interrumpida. **Entradas:** período, fecha de corte, cartera elegible e historial tarifario. **Salidas:** cargos creados/reutilizados y ejecución cerrada con totales e incidencias.

**P-E01 · Generación programada de cargos con tarifas históricas**

Escenario futuro · Procesamiento por lotes, importes conservados y reanudación sin duplicados




## Secuencia propuesta

1. El planificador registra una ejecución única por tipo/período, define el corte y obtiene una concesión temporal de ejecución para evitar dos coordinadores activos.
2. El procesador recupera el siguiente lote con orden estable y un punto de avance persistido. La política de elegibilidad establece desde qué fecha se cobra y el efecto de suspensión, baja o cambio de plan.
3. Resuelve para cada cliente la tarifa con vigencia en el período. Si no existe una fuente confiable, registra la incidencia sin inventar una tarifa histórica.
4. Crea solo cargos inexistentes. La restricción cliente/período impide duplicarlos y los cargos ya creados conservan su importe. Confirma el lote y su avance de forma coordinada.
5. Si quedan lotes, continúa. Si no quedan, cierra la ejecución y publica el resumen con generados, existentes y observados.

**Fallos y recuperación.** Si cae el trabajador, vence la concesión y otra ejecución autorizada retoma el avance confirmado; puede volver a leer el último lote sin duplicar cargos. Una incidencia de tarifa no detiene toda la cartera, pero impide declarar completitud total. Un error transitorio de infraestructura reintenta el lote con espera y un máximo configurable; al agotarlo queda la ejecución fallida y se alerta al operador.

**Decisiones de negocio previas.** Debe definirse si un cambio de plan rige desde el mes siguiente o si admite prorrateo. Se propone comenzar con vigencia por mes completo, sin prorrateo automático. Un cambio retroactivo no revaloriza cargos existentes: requiere un ajuste explícito y auditable. No se distribuye automáticamente crédito anterior como parte de este proceso; esa regla necesita una decisión financiera independiente.

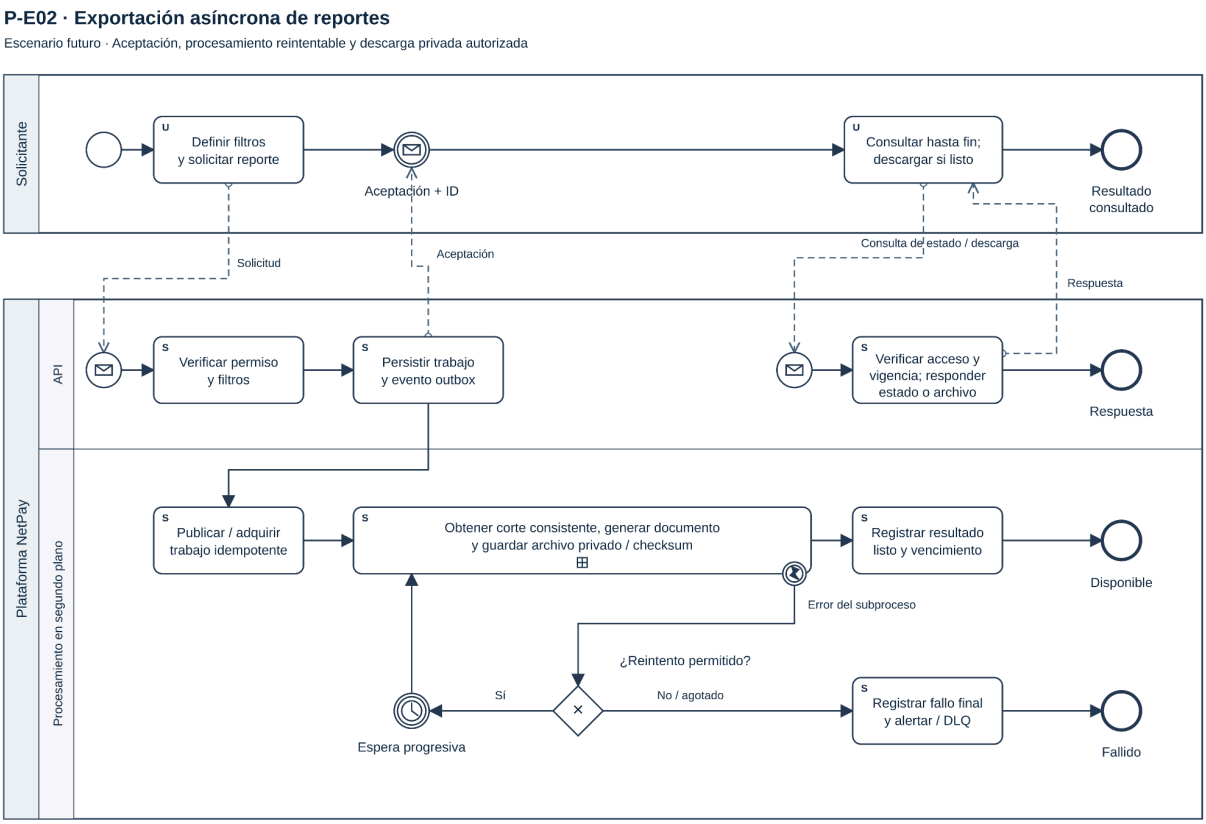
**Criterios de aceptación:** repetir una ejecución no duplica cliente/período; una caída tras confirmar un lote no pierde su avance; una tarifa nueva no cambia cargos existentes; toda fila elegible termina generada, existente u observada; la lectura ordinaria puede consultar obligaciones sin ejecutar una sincronización masiva. Las lecturas con efectos actuales se retiran solo después de acreditar cobertura equivalente.

**Aporte a escalabilidad:** distribuye trabajo por lotes, permite regular conexiones y paralelismo por particiones de cartera, y reduce escrituras inesperadas en consultas. Tener varios trabajadores exige concesiones y claves de unicidad; activar un temporizador en cada réplica de API no constituye coordinación.

## 4.3 P-E02 — Exportación asíncrona de reportes

**Objetivo:** permitir reportes extensos sin mantener abierta una solicitud HTTP ni degradar innecesariamente el registro de cobros.

**Participantes:** empleado solicitante y plataforma NetPay. **Carriles de plataforma:** API y procesamiento en segundo plano. **Entradas:** solicitante, permisos, filtros, formato y criterio de corte. **Salidas:** solicitud consultable y archivo privado disponible, o fallo registrado.




## Secuencia propuesta

1. El empleado define filtros y formato. La API verifica autorización y guarda trabajo y evento outbox en una misma transacción; devuelve aceptación e identificador público, no un documento terminado.
2. Un despachador publica el trabajo. El procesador adquiere su ejecución y reconoce entregas duplicadas para no producir resultados contradictorios.
3. Obtiene los datos según los filtros guardados, genera el archivo y lo coloca en almacenamiento privado. Después registra checksum, tamaño, formato, ubicación, corte y vencimiento.
4. Al finalizar, el solicitante consulta el estado desde la interfaz. Si el trabajo está listo, pide descarga; la API vuelve a verificar sus permisos y la propiedad/alcance del resultado.
5. El acceso puede realizarse por respuesta autorizada o por enlace firmado de corta duración. Al expirar la retención se elimina el objeto y se conserva el resultado administrativo permitido.

**Corte de datos.** La propuesta inicial toma los datos al comenzar la lectura del trabajador y registra ese instante en el archivo. La lectura consistente debe obtenerse en una transacción de lectura con aislamiento apropiado o materializando el conjunto de resultados; guardar una fecha por sí solo no congela la base. Se evita mantener una transacción abierta durante toda la

subida del archivo. Un snapshot exacto al solicitarlo necesitaría materialización o historia temporal adicional y no se asume implementado.

Si un reintento necesita repetir la lectura, registra su nuevo corte efectivo antes de publicar el resultado; si ya existe un conjunto materializado válido, lo reutiliza. No se presenta un archivo regenerado con la fecha de un snapshot que nunca se conservó ni se expone una salida parcial como definitiva.

**Fallos y recuperación.** Los errores transitorios admiten reintentos limitados y espera progresiva. Al agotarlos, el trabajo termina fallido y el mensaje se dirige a una cola de fallos para intervención. Si el archivo se cargó pero faltó confirmar el estado SQL, el reintento reconoce su clave/checksum y finaliza de forma idempotente; una limpieza periódica elimina objetos huérfanos. Un fallo de notificación no debe volver a generar un reporte listo.

**Criterios de aceptación:** una entrega duplicada no crea dos resultados vigentes; cerrar el navegador no cancela el trabajo confirmado; el archivo respeta filtros y corte; un usuario sin permiso vigente no lo descarga; el estado final y el vencimiento son consultables; la carga de exportación puede limitarse sin bloquear todos los hilos de la API.

**Aporte a escalabilidad:** separa CPU/memoria documental de las transacciones interactivas. La cola amortigua picos, pero no elimina el costo SQL: se limita el número de exportaciones concurrentes y se miden sus consultas. Los formatos actuales son PDF y XLS binario; incorporar OOXML `.xlsx` es una evolución de exportador/contrato y no exige por sí sola tablas adicionales.

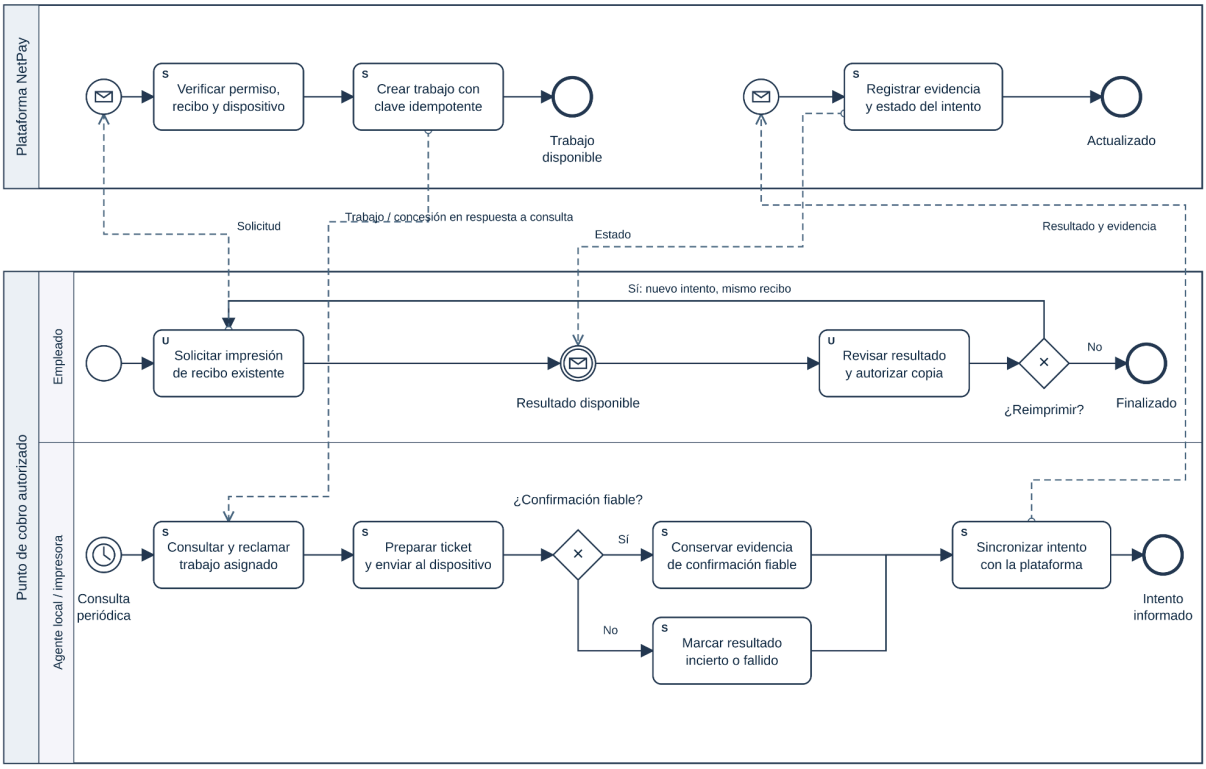
## 4.4 P-E03 — Impresión térmica distribuida y reimpresión controlada

**Objetivo:** entregar un recibo físico desde diferentes puntos de cobro manteniendo el pago confirmado, la identidad del comprobante y evidencia de cada intento de impresión.

**Participantes:** punto de cobro (empleado y agente local) y plataforma NetPay. **Disparador:** empleado solicita imprimir un recibo existente. **Entradas:** identificador del recibo, dispositivo autorizado y perfil de papel. **Salidas:** resultado de impresión confirmado o incierto/fallido, con PDF disponible como alternativa de entrega.

**P-E03 · Impresión térmica distribuida y reimpresión controlada**

Escenario futuro · Mismo recibo, agentes por punto de cobro y evidencia de cada intento




## Secuencia propuesta

1. El empleado selecciona un recibo válido y solicita imprimirlo. La API comprueba permiso, pertenencia del recurso y dispositivo autorizado; crea un trabajo con clave de idempotencia y contenido/versionado del comprobante.
2. El agente del punto de cobro consulta trabajos asignados mediante una conexión saliente autenticada y obtiene una concesión sobre el trabajo. El servidor no abre una conexión desde Internet hacia el USB del empleado.
3. El agente adapta el contenido al ancho útil y capacidades de la impresora, lo envía al controlador o mediante ESC/POS y registra el intento localmente antes de informar el resultado.
4. Cuando puede obtener un resultado fiable del dispositivo y/o una confirmación operativa del empleado, comunica la evidencia. La plataforma registra el nivel de confirmación y actualiza el trabajo.
5. Si el resultado es ambiguo, se marca para revisión. El empleado comprueba si salió papel antes de autorizar una reimpresión; esta crea un intento auditable y conserva el mismo número de recibo.

**Fallos y recuperación.** Falta de papel, impresora desconectada o corte de red no anulan el pago ni crean otro recibo. El agente conserva el identificador del intento para sincronizarlo al

recuperar conexión. Si se perdió la respuesta después de enviar el ticket, no se reenvía a ciegas: puede haberse impreso. Una solicitud aceptada por el spooler solo acredita encolado, no entrega física.

La repetición técnica conserva la clave de la solicitud original. Una copia deliberada requiere una nueva solicitud autorizada y un motivo, vinculados al mismo recibo; de ese modo la idempotencia no impide una reimpresión legítima ni la confunde con un doble envío accidental.

**Criterios de aceptación:** dos solicitudes con la misma clave no crean dos trabajos; un equipo no asignado no consume el trabajo; una reimpresión no duplica pagos/recibos; se distinguen encolado, envío, confirmación y resultado incierto; la desconexión después de enviar no provoca un duplicado automático; PDF e impresión contienen el mismo número e importe.

**Aporte a escalabilidad:** cada punto atiende su dispositivo local y la plataforma coordina trabajos y auditoría. La impresora lenta no ocupa una transacción de pago ni bloquea otras sucursales. Se reconoce que no es posible garantizar exactamente una salida física frente a una respuesta perdida sin soporte verificable del dispositivo y un procedimiento operativo.

# 5. Migraciones necesarias para evolucionar los datos

## 5.1 Base vigente y convenciones

El modelo actual incluye empleados y credenciales, roles/permisos con vigencia, clientes asociados a un plan y cobrador, períodos mensuales, cargos, pagos, aplicaciones pago/cargo, recibos y auditoría. V21 añade preferencia 2FA y desafíos de login. Los cargos conservan `expected_amount`; cliente/período, pago/cargo y recibo/pago tienen restricciones de unicidad.

Las migraciones siguientes son **propuestas de diseño**, no scripts ejecutados ni números Flyway reservados. Los identificadores M-01 a M-09 sirven para planificar entregas. Al implementarlas se tomará la próxima versión libre posterior al historial real. Nunca se reescribe una migración aplicada. Las nuevas tablas usan claves internas para relaciones y UUID públicos en contratos; no se introducen catálogos tipológicos sin una decisión funcional explícita.

| Propuesta | Estructura/datos necesarios | Integridad, transición y validación | Proceso |
|---|---|---|---|
| **M-01. Historial de contratación y tarifa** | Historial por cliente con plan, importe acordado, moneda, inicio/fin de vigencia y fuente; referencia | Evitar intervalos solapados, definir vigencia mensual y preservar importe de cargos existentes. Incorporar historia antigua solo con evidencia; marcar lo desconocido | P-E01 |

| Propuesta | Estructura/datos necesarios | Integridad, transición y validación | Proceso |
|---|---|---|---|
| | opcional desde cargo a la versión tarifaria | | |
| **M-02. Ejecuciones y lotes de cargos** | Cabecera de ejecución con tipo/período/corte/estado; lotes, punto de avance, contadores, concesión con vencimiento e incidencias por cliente | Un coordinador vigente por ejecución, clave única de ejecución y cliente/período único. Confirmación coordinada de lote/avance; reejecución sin duplicados | P-E01 |
| **M-03. Integridad concurrente financiera** | Bloqueo de cartera/cargos o campo de versión según estrategia; índices de consulta y, si aplica, registro de solicitud de pago con idempotencia | Serializar o detectar conflictos y reintentar la transacción completa. Comprobar sobreasignación existente antes del cambio; unicidad pago/cargo no basta entre pagos distintos | P-E01 y cobro |
| **M-04. Outbox e idempotencia de consumidores** | Evento con UUID, agregado, tipo, referencia de trabajo, fecha, estado de despacho e intentos; registro de consumo/efecto cuando sea necesario | Crear evento y solicitud en la misma transacción. Publicación repetible, consumo idempotente, bloqueo/concesión de despacho y retención acotada | P-E02; base reutilizable para otros trabajos |
| **M-05. Solicitudes de exportación** | UUID, solicitante, filtros inmutables, formato, corte, estado, intentos, concesión, checksum, objeto privado, tamaño, error y vencimiento | Una salida vigente por trabajo; revalidar acceso al descargar. Expiración coordinada entre fila y objeto; datos materializados si se exige snapshot estricto | P-E02 |
| **M-06. Dispositivos y trabajos de impresión** | Dispositivo con punto/identidad/perfil/capacidades; trabajo con recibo, clave de solicitud y estado; intentos con fechas, evidencia y motivo de reimpresión | Claves públicas, concesión por agente y solicitud idempotente. Separar envío/confirmación/incertidumbre; conservar recibo único por pago | P-E03 |
| **M-07. Resumen de importación por lote** | Cabecera, archivo/checksum, recurso, solicitante, estado y | Reanudar filas sin duplicar pagos; preservar `import_key` actual. | Incorporación masiva futura |

| Propuesta | Estructura/datos necesarios | Integridad, transición y validación | Proceso |
|---|---|---|---|
| | resultado de filas con clave de origen | Errores sin contraseñas y archivo con acceso/retención controlados | |
| **M-08. Normalización de identidad y fechas** | Conciliación de correos y posible índice único parcial normalizado para empleados vigentes; instantes técnicos con zona explícita | Resolver duplicados antes de restringir. Determinar la zona de datos previos antes de convertir; mantener `DATE` para fecha efectiva del pago | Transversal |
| **M-09. Acceso histórico y volumen** | Índices derivados de planes medidos; política de archivo; particionado solo si lo justifica el volumen | Comparar planes/tiempos, revisar FKs y unicidad antes de particionar; mantener consulta trazable de historia | Reportes y auditoría |

M-03 requiere una decisión explícita: si se usa bloqueo pesimista debe fijarse un orden estable para evitar interbloqueos; si se usa control optimista deben detectarse conflictos y reintentar toda la operación. Un campo de versión que el flujo de escritura no verifica no ofrece protección. La idempotencia debe reconocer una repetición de solicitud sin confundir dos cobros legítimos del mismo importe.

En M-06, `printed_at` actual no basta para conservar varios intentos o su resultado. Se mantendría como dato compatible durante la transición y se añadiría evidencia detallada. Su valor histórico no se transformaría en una confirmación física inexistente. Los estados propuestos de trabajos son técnicos y requieren definición/migración propia; no equivalen automáticamente a estados comerciales del catálogo actual.

## 5.2 Transición de datos sin reinterpretar el pasado

1. **Inventariar:** confirmar versión Flyway aplicada, restricciones, duplicados, volúmenes y contratos frontend/backend.
2. **Respaldar y ensayar:** obtener copia externa verificada y ejecutar restauración aislada antes de cambios de datos significativos.
3. **Expandir:** crear tablas/columnas nuevas compatibles y permitir referencias opcionales durante la transición.
4. **Completar datos:** cargar historia respaldada por fuentes; registrar filas ambiguas y no reemplazar importes conservados por el precio actual.
5. **Cambiar lectores/escritores:** habilitar el nuevo flujo en una cartera o grupo piloto, conciliando conteos y montos. Evitar dos procesos independientes generando obligaciones sin coordinación.

6. **Verificar:** probar duplicados, fallos a mitad de lote, concurrencia, permisos, archivos expirados e impresión ambigua.
7. **Contraer:** retirar estructuras y sincronizaciones antiguas solo cuando no queden consumidores y se haya cerrado el período de compatibilidad.

El rollback de una imagen no revierte una transformación SQL ni una impresión ya enviada. Para cambios compatibles puede volver el binario anterior; para cambios incompatibles se decide corrección hacia adelante o recuperación de una copia conciliada. Las migraciones se ejecutan una sola vez de forma coordinada en la liberación, evitando que cada réplica sea el mecanismo habitual de actualización.

Migrar a RDS cambia la operación y ubicación de PostgreSQL, no exige rediseñar todas las tablas. El corte requiere transferir datos, validar extensiones y roles, detener/coordinar escrituras, comparar conteos/importes, cambiar conexión y conservar una vía de recuperación. Las secuencias, historial Flyway y activos externos también forman parte de la conciliación.

# 6. Tecnologías nuevas y justificación

## 6.1 Selección propuesta

| **Tecnología** | **Problema concreto que resuelve** | **Condición de adopción y costo operativo** |
|---|---|---|
| **Amazon RDS for PostgreSQL** | Separar datos de API, gestionar respaldos/PITR y facilitar operación de la base | Aprobar costo, red privada, retención, roles y restauración. Multi-AZ mejora disponibilidad; no es una réplica de lectura para repartir consultas |
| **S3 privado** | Archivos de trabajos, retención y copias fuera del host | Cifrado, permisos por prefijo, expiración y enlaces firmados cortos. Un bucket por sí solo no crea ni valida respaldos; Cloudinary puede mantenerse para sus activos actuales |
| **CloudFront + S3 estático** | Reducir administración del frontend y distribuir contenido | Revisar HTTPS, caché de HTML/assets, rutas hash, URL API y CORS; publicar |

| Tecnología | Problema concreto que resuelve | Condición de adopción y costo operativo |
|---|---|---|
| | | bundle nuevo cuando cambie configuración compilada |
| **SQS y cola de fallos (DLQ)** | Amortiguar picos y desacoplar reportes de HTTP | Consumidores idempotentes, timeout de visibilidad acorde al trabajo, reintentos limitados y alertas por edad/cola de fallos |
| **EventBridge Scheduler** | Iniciar generación periódica sin depender de una API concreta | Identidad restringida y ejecución única por período en la aplicación; los disparos pueden repetirse. En un piloto pequeño puede bastar un planificador único controlado |
| **ECS con Fargate, de forma gradual** | Ejecutar API y trabajadores con escalado y reemplazo independientes | Preparar healthchecks, límites de recursos, red y costos. Para pocas cargas puede mantenerse EC2; no es requisito incorporar Kubernetes |
| **Application Load Balancer** | Distribuir tráfico HTTPS y retirar réplicas no disponibles | Base compartida, drenaje de conexiones, certificados adecuados y pruebas al perder una réplica |
| **CloudWatch + OpenTelemetry** | Detectar saturación y correlacionar fallos entre API/trabajadores | Definir métricas, alarmas, retención y redacción de datos; instalar instrumentación y probar notificaciones, no solo crear un panel |
| **pgBackRest, si PostgreSQL sigue autogestionado** | Copias físicas completas/diferenciales/incrementales y archivado WAL para recuperación temporal | Operar cadena, almacenamiento externo, retención y ensayos. Es una alternativa operativa a |

| Tecnología | Problema concreto que resuelve | Condición de adopción y costo operativo |
|---|---|---|
| | | respaldos administrados, no otra base de datos |
| **QZ Tray o agente local equivalente + controlador/ESC/POS** | Conectar la aplicación web con impresoras del punto de cobro | Validar soporte real del modelo, instalación, autenticación/firma de solicitudes, origen autorizado y costos de firma/soporte. Requiere equipo físico |
| **Redis, solo con evidencia de necesidad** | Reducir lecturas repetidas de datos tolerantes a caché | Medir beneficio, TTL e invalidación; no almacenar el único saldo ni conceder acceso con permisos obsoletos |

ESC/POS es una familia de comandos habitual en impresoras de tickets; su compatibilidad, juegos de caracteres y funciones de corte varían por fabricante. QZ Tray es un puente local, no un componente que se instala en EC2 para alcanzar el USB del empleado. La selección definitiva se hace con el modelo adquirido y su sistema operativo.

## 6.2 Cambios transversales del frontend y seguridad

Angular incorporaría una bandeja de trabajos con estado, error, vencimiento y descarga; la consulta de estado puede empezar con sondeo periódico moderado y evolucionar a eventos de servidor si se justifica. Debe diferenciarse “solicitud aceptada” de “archivo listo” e “impresión enviada” de “confirmada”. Los filtros guardados en el trabajo no dependen del estado visual que el usuario cambie después.

La API conserva autoridad sobre permisos y alcance. Un UUID difícil de adivinar no sustituye verificar la propiedad de un trabajo o recibo. Se controlan también la descarga firmada y la identidad del agente; las credenciales locales son revocables y acotadas al punto asignado. El segundo paso por correo actual puede mantenerse; una futura identidad administrada/TOTP requiere planificación de recuperación de cuentas y no es una condición para los tres procesos propuestos.

La automatización puede evolucionar a credenciales temporales mediante OIDC entre GitHub y AWS, si la cuenta lo permite, reduciendo claves persistentes. Cada componente conserva imágenes inmutables y evidencia de la versión. Las nuevas dependencias se adoptan por etapa y presupuesto; no se plantea activar toda la tabla simultáneamente.

# 7. Ampliación pendiente: impresora térmica para recibos

## 7.1 Motivo de no implementación y capacidad entregada

**La integración con impresora térmica no se pudo implementar ni validar físicamente porque el cliente no pudo comprar el equipo durante el desarrollo.** Desde el planteamiento inicial esta ampliación estaba condicionada a disponer de una impresora compatible; la generación de recibos digitales se mantuvo como solución principal.

Actualmente existe un recibo asociado al pago completado, con número único, importe, cliente y fecha de emisión. Se puede generar o recuperar su PDF; la interfaz permite abrirlo y registrar una marca de impresión. Esa marca refleja una acción de la aplicación, no una respuesta de un dispositivo ni una garantía de que se entregó papel. No se acredita conexión USB/red, corte de papel, selección de ancho ni pruebas de calidad sobre una impresora térmica real.

## 7.2 Cómo podría haberse integrado

**Primer nivel: impresión asistida del PDF.** Una vez adquirido el dispositivo se instalaría el controlador del fabricante en el equipo de cobro, se configuraría como impresora del sistema y se prepararía una plantilla de recibo para el ancho útil. El empleado abriría el PDF y seleccionaría la térmica en el diálogo de impresión. Es una ampliación de menor complejidad, pero depende de configuración local, escala, márgenes y confirmación manual; abrir el diálogo no confirma impresión.

**Segundo nivel: impresión directa controlada.** Si se necesita automatizar salida, corte y elección de dispositivo, un puente/agente local autenticado recibiría trabajos autorizados, convertiría los datos del recibo a formato del controlador o ESC/POS y gestionaría su envío. La API conservaría el registro central de trabajos y resultados del proceso P-E03. No se enviaría un PDF arbitrario como si fuera una secuencia ESC/POS: se renderiza con el controlador compatible o se genera una representación específica del ticket.

El navegador no dispone de acceso universal y silencioso a cualquier impresora USB. WebUSB/Web Serial dependen del navegador, permisos y soporte del dispositivo y no constituyen una solución general sin piloto. La EC2 tampoco tiene acceso directo a la red/USB de cada cobrador. Para varios puntos se prefiere un agente con conexión saliente, sin publicar puertos de impresora en Internet. La impresión de recibos ya registrados puede reanudarse tras una desconexión; registrar cobros completamente sin Internet sería un proyecto adicional de sincronización, fuera de esta propuesta.

## 7.3 Contrato documental y diseño del ticket

PDF y ticket deben provenir del mismo recibo y conservar número, fecha, cliente, importe, moneda y referencia al pago. Se propone guardar una versión del contenido emitido o una instantánea de sus datos relevantes para que una reimpresión no cambie al editar después el nombre/contacto del cliente. Esto no crea un segundo comprobante financiero.

Antes del diseño se debe confirmar papel de 58 u 80 mm, ancho imprimible real, resolución, interfaz USB/Ethernet/Bluetooth, controlador, emulación y caracteres soportados. Se probarían nombres largos, tildes, símbolo monetario, importes grandes, saltos y corte. Un QR podría abrir una verificación protegida del comprobante, pero no contendría JWT, credenciales ni datos personales innecesarios. El recibo administrativo no se convierte en factura fiscal por imprimirse en otro soporte.

## 7.4 Evidencia e incertidumbre física

| **Evidencia disponible** | **Lo que permite afirmar** |
|---|---|
| Solicitud recibida por API | Existe un trabajo autorizado |
| Aceptación del controlador/spooler | El sistema local aceptó la cola, no necesariamente imprimió |
| Envío al puerto/dispositivo | Se transmitieron datos; una respuesta perdida puede dejar resultado ambiguo |
| Estado verificable del dispositivo | Se conoce el nivel reportado por ese modelo; debe conservarse su significado |
| Confirmación visual del empleado | Existe una constatación operativa de salida del ticket |

Se conservarían eventos e intentos en lugar de una sola bandera. Reimprimir exige permiso y motivo; mantiene el mismo número y puede rotularse “COPIA”. La reimpresión deliberada es distinta del reintento técnico. Los resultados inciertos se revisan antes de reenviar, porque la idempotencia de la base no impide por sí sola que salgan dos papeles.

## 7.5 Piloto necesario para aceptar la integración

1. Adquirir y registrar el modelo, interfaz, controlador, papel y punto de operación.
2. Comparar un conjunto de recibos PDF/ticket: contenido, moneda, legibilidad, caracteres y corte.

3. Probar sin papel, tapa abierta, desconexión, reinicio y pérdida de respuesta después del envío.
4. Verificar permisos, agente equivocado, solicitud repetida y reimpresión con motivo.
5. Comprobar que fallar la impresión no altera el pago, el saldo ni el número de recibo.
6. Medir tiempo y tasa de error; documentar instalación, recuperación y reposición de consumibles antes de extender a otros puntos.

Hasta completar ese piloto, la impresión térmica sigue siendo una ampliación diseñada y pendiente. El PDF permite conservar y entregar el comprobante durante la transición.

# 8. Continuidad, validación y prioridades de entrega

## 8.1 Protección de información

El volumen EBS cifrado protege persistencia local, pero no constituye una copia externa. Se propone cumplir la política de copia completa semanal, incremental diaria y archivado transaccional cada 15–30 minutos, o configurar una protección administrada equivalente y aprobada. En PostgreSQL autogestionado, WAL es el registro de transacciones necesario para recuperar a un instante dentro de una cadena válida; un `pg_dump` aislado no es un incremental ni sustituye ese archivado.

La retención, cifrado, permisos y frecuencia deben acompañarse de una restauración en entorno aislado y conciliación de cargos, pagos, aplicaciones y recibos. También se inventarían documentos externos, configuración recuperable y claves necesarias, evitando depender del único host afectado. Una réplica o Multi-AZ mejora continuidad, pero puede replicar una eliminación accidental; no sustituye respaldos recuperables.

**RPO** es la pérdida temporal máxima de datos tolerada; **RTO**, el tiempo máximo de recuperación tolerado. Ambos deben acordarse y medirse con Gravity. Una frecuencia de copia propuesta no demuestra que esos objetivos se cumplan.

## 8.2 Entregas y evidencia de aceptación

| **Prioridad** | **Entrega acotada** | **Evidencia para avanzar** |
|---|---|---|
| 1 | Conciliación de reglas, acceso por registro y observabilidad | Cifras correctas, casos de acceso ajeno rechazados y métricas de línea base |
| 2 | Copias externas y recuperación | Restauración medida, checksum, datos conciliados y responsables |
| 3 | Base independiente y concurrencia financiera | Prueba de corte/recuperación y cobros simultáneos sin sobreasignación |

| Prioridad | Entrega acotada | Evidencia para avanzar |
|---|---|---|
| 4 | Historial tarifario y P-E01 | Lotes reanudables, cobertura de cartera y cargos históricos conservados |
| 5 | Outbox, trabajos y P-E02 | Reportes repetibles, acceso controlado, reintentos limitados y cobro estable bajo carga |
| 6 | CDN y API replicada según medición | Pruebas de caché/HTTPS y continuidad al retirar una réplica |
| Condiciona da al equipo | M-06 y piloto P-E03 | Ticket real validado, fallos físicos controlados y reimpresiones trazables |

La fecha de cada entrega depende de disponibilidad del equipo, presupuesto, volumen y capacidad de soporte. Los datos usados para las pruebas deben representar importes parciales/completos, cambios de tarifa, anulaciones, carteras de varios cobradores y períodos vencidos; una demostración con listas vacías no valida integridad bajo crecimiento.

## 9. Resultado esperado de la evolución

La solución evolucionaría desde un despliegue pequeño hacia una operación con datos protegidos, procesos reanudables y recursos dimensionables por carga. La generación de obligaciones dejaría de depender de consultas pesadas; los reportes extensos se ejecutarían sin mantener conexiones interactivas; y cada punto podría imprimir recibos sin acoplar la confirmación financiera a un dispositivo físico.

El principio de continuidad es conservar una única explicación del negocio: el pago representa dinero recibido, el cargo una obligación, la aplicación su cobertura y el recibo su comprobante. Las migraciones, nuevas tecnologías y procesos propuestos se aceptan cuando mantienen esa coherencia y muestran mejoras medibles de capacidad, recuperación y operación. La impresión térmica permanece pendiente por falta del equipo del cliente, con una ruta técnica definida y el PDF como capacidad disponible.